INTERNET OF THINGS

Unit 2
Microcontrollers and Interfacing

Complete Easy-Language Study Notes
Arduino • ATmega328P • ADC • Programming • ESP8266

Source: unit-2.pptx (DU #2504CS323)
Prepared as an exam-ready revision resource

Table of Contents

Placeholder for table of contents 0

How to use these notes

These notes follow the order and topics of the supplied Unit 2 presentation. Each topic begins with a simple definition, then explains the idea in easy words, followed by key points, examples, comparisons, code explanations, exam points, and quick revision material where useful. The final chapter contains a complete revision pack and practice questions.

1. Microcontroller Basics

1.1 What is a Microcontroller?

Simple definition: A microcontroller is a small computer on a single chip. It contains a processor, memory, input/output pins, and support circuits such as clock and reset circuits.

A microcontroller is used to control a device automatically. For example, it can read a temperature sensor and switch a fan ON or OFF. The word “micro” means small, and “controller” means a device that controls something.

Detailed explanation: A normal computer needs separate parts such as a processor, RAM, storage, and input/output devices. A microcontroller combines most of these parts inside one small chip. Therefore, it is cheap, small, and suitable for embedded systems.

Embedded system: An embedded system is a computer system built inside another device to perform one or a few specific tasks. Examples include a washing machine, air conditioner, microwave oven, and car engine-control system.

Main components of a microcontroller architecture:

- **CPU and ALU:** The Central Processing Unit executes instructions. The Arithmetic and Logic Unit performs calculations such as addition, subtraction, AND, OR, and comparison.
- **Clock and timing circuit:** Provides timing signals so that all parts work in a fixed sequence.
- **Reset control:** Starts the microcontroller from a known initial state when power is applied or the reset button is pressed.
- **Power distribution:** Supplies proper voltage to internal parts.
- **Control store / program memory:** Stores the program or firmware, usually in ROM or Flash memory.
- **RAM:** Temporarily stores variables and data while the program runs.
- **Input/output ports:** Connect the microcontroller to sensors, buttons, LEDs, motors, and other devices.

Block diagram explanation: The CPU is at the centre. Program memory gives instructions to the CPU. RAM stores temporary data. The clock gives timing, reset control restarts the system, and I/O ports connect to the outside world.

Examples of 8-bit microcontrollers:

- Motorola 6811
- Intel 8051
- Zilog Z8
- Microchip PIC

The Intel 8051 family has many suppliers, including Intel, ATMEL, Philips/Signetics, AMD, Infineon, and Dallas Semiconductor/Maxim.

1.2 Memory Architecture

Microcontrollers use two main memory architectures: Harvard and Princeton (Von Neumann).

Harvard architecture

Simple definition: In Harvard architecture, program memory and data memory are separate.

The processor uses separate paths to read instructions from program memory and read/write data in data memory. Because both memories can be accessed at the same time, instructions can execute in fewer cycles than in Von Neumann architecture.

- Program ROM stores the program code.
- Variable RAM stores data.
- Processor stack stores return addresses and temporary values.
- Advantage: faster instruction execution because instruction and data buses are separate.

Princeton / Von Neumann architecture

Simple definition: In Princeton or Von Neumann architecture, program and data are stored in one common memory space.

The same memory interface is used for program code, variables, and stack data. This makes memory interfacing and management simpler, but instruction and data cannot always be fetched at the same time.

Feature	Harvard architecture	Princeton / Von Neumann architecture
Memory arrangement	Separate program and data memory	Common memory for program and data
Buses	Separate instruction and data buses	Common bus/interface
Speed	Generally faster	Generally slower for simultaneous access
Complexity	More complex hardware	Simpler memory interfacing
Example	ATmega328P used in Arduino Uno	Traditional general-purpose computer design

1.3 Microcontroller Memory Types

Control store: This is program memory or firmware memory. It stores the application program and low-level code required to run the device. It is non-volatile ROM/Flash memory, which means the program remains even when power is removed.

Variable area: This stores temporary data while the program runs. It includes four types of storage:

- **Bits:** Single binary values, either 0 or 1.
- **Registers:** Small, fast storage locations inside the CPU.
- **Variable RAM:** Memory used for program variables.
- **PC stack:** Part of RAM that works as Last-In-First-Out (LIFO) memory. It stores return addresses during function calls and must be initialised with a starting stack address.

1.4 Microcontroller vs Microprocessor

Parameter	Microprocessor	Microcontroller
Basic role	Heart of a computer system	Heart of an embedded system
Memory and I/O	Connected externally	Available internally on the chip
Circuit complexity	More complex because external components are needed	Less complex because memory and I/O are on-chip
Compact systems	Not suitable for compact systems	Suitable for compact systems

Parameter	Microprocessor	Microcontroller
Efficiency	Less efficient for simple control tasks	More efficient for dedicated control tasks
Applications	Personal computers	Washing machines, air conditioners, embedded devices

Easy way to remember: A microprocessor is only the “brain”; other parts must be connected outside. A microcontroller is a brain with memory and input/output already built in.

Exam points

- A microcontroller is a single-chip computer.
- CPU, ALU, RAM, ROM/Flash, clock, reset, and I/O are its main parts.
- Harvard architecture uses separate program and data memory.
- Von Neumann architecture uses common memory.
- ROM/Flash is non-volatile; RAM is volatile.
- Microcontrollers are used in embedded systems.

Quick revision summary

A microcontroller is a small, complete controlling computer on one chip. It contains a CPU, memory, clock, reset circuit, and I/O ports. Harvard architecture separates program and data memory, while Von Neumann architecture uses one common memory. Microcontrollers are compact and efficient for embedded control applications.

2. Analog and Digital Signals

2.1 Analog Signal

Simple definition: An analog signal is a continuous signal that changes smoothly over time and can take any value within a range.

For example, room temperature does not jump from 25°C to 26°C instantly; it changes gradually through values such as 25.1°C, 25.2°C, and so on. Similarly, a microphone produces a continuously changing voltage according to sound.

- It has infinite possible values in a range.
- Its graph is smooth and wavy.
- Examples: sound waves, temperature, microphone voltage, light intensity.

2.2 Digital Signal

Simple definition: A digital signal is a discrete signal that has only fixed values, usually 0 and 1.

In electronics, 0 usually means LOW voltage, such as 0 V, and 1 means HIGH voltage, such as 5 V. Digital systems use binary numbers, so they can store, process, and transmit information reliably.

- It has a limited set of values, commonly 0 and 1.
- Its graph appears as square steps.
- Examples: computer data, binary numbers, digital audio.

Point	Analog signal	Digital signal
Nature	Continuous	Discrete
Values	Infinite values in a range	Fixed values, usually 0 and 1
Graph	Smooth wavy curve	Square steps
Noise effect	More affected by noise	Less affected by noise
Examples	Sound, temperature, microphone output	Computer data, digital audio, binary signals

2.3 Why Analog-to-Digital Conversion is Needed

Microcontrollers are digital devices. They understand only binary values, not continuously changing analog voltages. However, most real-world sensors give analog output. Therefore, an ADC is required.

Reason	Explanation
Microcontrollers are digital	They cannot directly understand analog signals
Sensor interfacing	Temperature, pressure, and sound sensors often give analog output
Digital storage	Data must be digital to store and process it in memory
Communication	Digital data is easier to transmit over long distances
Accuracy and noise reduction	Digital signals are less affected by noise than analog signals

2.4 ADC: Analog-to-Digital Converter

Simple definition: An ADC is a circuit that converts a continuous analog voltage into a digital binary number.

Working steps:

1. **Sampling:** The ADC measures the analog signal at fixed time intervals. For example, it may take one voltage reading every 1 millisecond.
2. **Quantization:** Each sampled value is approximated to the nearest allowed digital level. An 8-bit ADC has 256 levels.
3. **Encoding:** The selected level is converted into a binary number.
4. **Output:** The microcontroller receives and processes the binary value.

Example workflow: An analog input from 0 V to 5 V enters a 10-bit ADC. The output can be any binary number from 0000000000 to 1111111111, which means decimal 0 to 1023.

ADC resolution formula

For an n-bit ADC, the number of digital levels is 2^n. The step size is:

```
Step size = Reference voltage range / Number of levels
```

For a 10-bit ADC with a 5 V reference:

```
Number of levels = 2^10 = 1024
Step size = 5 V / 1024 = 0.00488 V = 4.88 mV
```

So, each ADC count represents approximately 4.88 mV. If the ADC reading is 512, the approximate input voltage is:

```
Voltage = (ADC value / 1023) × 5 V
Voltage = (512 / 1023) × 5 = 2.50 V
```

Note: Arduino Uno has a 10-bit ADC, so analogRead() returns values from 0 to 1023. The highest value 1023 corresponds to the reference voltage, normally 5 V on the Uno.

Exam points

- Analog signals are continuous; digital signals are discrete.
- ADC converts analog signals to digital form.
- ADC works through sampling, quantization, and encoding.
- A 10-bit ADC gives 1024 levels, with readings 0 to 1023.
- Digital signals are less affected by noise.

Quick revision summary

Real-world quantities such as temperature, sound, and light are analog. Microcontrollers work digitally. An ADC samples an analog signal, converts each sample to the nearest digital level, and encodes it in binary. Arduino Uno uses a 10-bit ADC, giving readings from 0 to 1023.

3. Arduino and Arduino Uno

3.1 What is Arduino?

Simple definition: Arduino is an open-source electronics platform that includes easy-to-use hardware boards and software for programming them.

Arduino has two main parts:

- **Hardware:** The Arduino board contains a microcontroller and supporting circuits. We send instructions to the microcontroller to control devices.
- **Software:** The Arduino IDE is used to write, compile, and upload programs. The Arduino programming language is based on Wiring, and the IDE is based on the Processing tool.

Arduino boards can read inputs such as light from a sensor, a button press, or an online message. They can then produce outputs such as turning an LED ON, running a motor, or publishing data online.

3.2 Why Use a Development Board?

A microcontroller chip has many pins, but by itself it does not provide ready-made ports, USB connection, voltage regulation, or other peripheral support. To use a bare chip, we would need to design an electronic circuit.

A development board solves this problem by providing ready-made circuits around the microcontroller. It allows us to program and connect devices without designing the complete supporting circuit ourselves.

- It provides power supply circuits.
- It provides USB communication.
- It provides input/output headers.
- It makes programming easy.
- It saves circuit-design time and effort.

3.3 Arduino Uno and ATmega328P

The Arduino Uno is an open-source microcontroller board developed by Arduino.cc. It is based on the Microchip ATmega328P microcontroller, originally manufactured by Atmel.

ATmega328P architecture:

- **Processor type:** 8-bit RISC processor.
- **Memory architecture:** Harvard architecture.
- **Native data size:** 8 bits, i.e., 1 byte.
- **Addressing:** 16-bit data addressing.
- **SRAM:** 2 KiB, used to store running data.
- **Flash memory:** 32 KiB, used to store program code.
- **EEPROM:** 1 KiB, used for small permanent data.
- **Communication and I/O:** Digital I/O, analog input, serial/parallel communication, timers, interrupts, ADC, analog comparator, and UART.

Block diagram explanation: The CPU is connected to Flash, SRAM, EEPROM, interrupts, I/O lines, ADC, analog comparator, UART, and timer/counter. Flash stores the program, SRAM stores temporary data, EEPROM stores small permanent data, and ADC converts analog inputs to digital values.

3.4 Arduino Shields

Simple definition: An Arduino shield is an additional board that is mounted directly on top of an Arduino board to add new features.

Why shields are useful:

- They add new functions, such as motor control, Ethernet, Wi-Fi, or display support.
- They can be attached and removed easily.
- They avoid complex wiring.
- Their hardware components are easy to use.

3.5 Bootloader and Firmware

Bootloader: A bootloader is a small program that helps load the main program into the microcontroller. In general computers, it helps load the operating system after startup.

In Arduino, the AVR bootloader allows programming or re-programming of the AVR microcontroller through the PC serial port instead of using a separate hardware programmer. Once written, it remains in the chip until the chip is erased.

Firmware: Firmware is permanent software stored in read-only memory. In Arduino, firmware includes the bootloader and the user program, also called a sketch.

- The USB-TTL chip contains firmware for USB communication.
- The main ATmega328P chip contains a bootloader.
- The ATmega328P also stores the user sketch, such as an LED-blink program.

3.6 Arduino Uno Block Diagram

The Arduino Uno contains these main parts:

- **Power supply:** Provides 5 V and 3.3 V to the board and external devices.
- **USB port:** Used for communication with a computer and for powering the board.
- **USB-TTL interface:** Converts USB signals to serial TTL signals. On the Uno, the ATmega16U2 chip performs this function.
- **ATmega328P microcontroller:** The main brain of the board; it stores and runs the user program.
- **Digital I/O pins:** Pins 0 to 13.
- **PWM pins:** Pins marked with (~): 3, 5, 6, 9, 10, and 11.
- **Analog input pins:** A0 to A5.
- **ADC:** 10-bit analog-to-digital converter.
- **Serial communication:** Uses pins 0 (RX) and 1 (TX).

3.7 Important Arduino Uno Components

Component	Purpose
Reset button	Restarts the program loaded on the Arduino board
AREF	Analog Reference pin; used to set an external reference voltage
Ground pins	Provide 0 V reference for circuits

Component	Purpose
Digital I/O pins 0–13	Can be used as digital inputs or outputs
PWM pins (~)	Can simulate analog output using PWM
USB connection	Powers the board and uploads sketches
TX/RX LEDs	Show transmit and receive activity
ATmega328P	Main microcontroller; stores and executes the program
Power LED	Turns ON when the board receives power
Voltage regulator	Controls the voltage supplied to the board
DC power barrel jack	Allows powering the board using an external DC supply
3.3 V pin	Supplies 3.3 V to external components
5 V pin	Supplies 5 V to external components
Analog pins A0–A5	Read analog sensor signals through the ADC

3.8 Power Supply

The Arduino Uno can be powered in three common ways:

1. Through a USB cable connected to a computer.
2. Through a 9 V battery pack for portable projects.
3. Through a 9 V DC power supply connected to the barrel jack.

3.9 Digital and Analog Pins

- The Uno has 14 digital I/O pins.
- Six digital pins support PWM: 3, 5, 6, 9, 10, and 11.
- The Uno has six analog input pins: A0 to A5.
- Analog pins use a 10-bit ADC and return values from 0 to 1023.
- Analog pins can also be used as digital I/O pins.

3.10 Reset, Crystal, and LEDs

Reset button: It is connected to the PC6/Reset pin of the ATmega328P. The pin is pulled up through a 10 kΩ resistor. When the button is pressed, the pin is pulled to ground and the microcontroller resets.

Crystal oscillator / ceramic resonator: A microcontroller needs a clock source to work. The clock decides how fast the microcontroller executes instructions. The Uno uses a 16 MHz crystal oscillator for the ATmega16U2 chip and a 16 MHz resonator for the ATmega328P.

On-board LEDs: The Uno has four LEDs: one power indicator, two TX/RX activity LEDs, and one LED connected to digital pin 13. The pin-13 LED is useful for testing the board.

3.11 ICSP Pins

ICSP means In-Circuit Serial Programming. The Uno has two 6-pin ICSP connectors:

- One connector programs the USB-TTL firmware into the ATmega16U2 chip.
- The other connector burns the bootloader into the ATmega328P microcontroller.

Exam points

- Arduino Uno uses the ATmega328P microcontroller.
- It has 14 digital I/O pins and 6 analog input pins.
- PWM pins are 3, 5, 6, 9, 10, and 11.
- It has a 10-bit ADC.
- Flash is 32 KiB, SRAM is 2 KiB, and EEPROM is 1 KiB.
- The bootloader allows program upload through USB.
- ICSP is used for low-level programming of the chips.

Quick revision summary

Arduino is an open-source hardware-plus-software platform. The Uno is built around the ATmega328P, which has 32 KiB Flash, 2 KiB SRAM, 1 KiB EEPROM, 14 digital pins, 6 analog inputs, and a 10-bit ADC. The USB port powers the board and uploads sketches through the USB-TTL interface and bootloader.

4. Arduino IDE and Programming Basics

4.1 Arduino IDE

IDE means Integrated Development Environment. The Arduino IDE is the software used to write, check, compile, and upload Arduino programs.

Main parts of the Arduino IDE:

- **Code editor:** The area where we write the program.
- **Text console:** Displays messages such as errors and compilation progress.
- **Toolbar:** Contains common buttons such as Verify, Upload, New, Open, Save, and Serial Monitor.
- **Menu bar:** Contains menus for files, editing, tools, board selection, and port selection.

Toolbar button	Function
Verify	Checks the code for errors by compiling it
Upload	Compiles the code and uploads it to the selected board
New	Creates a new sketch
Open	Opens an existing sketch from the sketchbook
Save	Saves the current sketch
Serial Monitor	Opens the window used to view data sent by Arduino

4.2 Compiling and Uploading a Program

To compile a program, click **Verify**. The status bar shows “Compiling sketch”. If there is no error, it shows “Done compiling”.

To upload a program:

1. Go to **Tools > Board** and select the correct Arduino board.
2. Go to **Tools > Port** and select the serial port of the board, often COM3 or higher on Windows.
3. Click **Upload**.
4. During uploading, the TX and RX LEDs flash.
5. When complete, the IDE shows “Done uploading”.

4.3 Basic Arduino Program Structure

```
void setup() {
// setup code: runs once
}

void loop() {
// main code: runs repeatedly
}
```

setup() function: It runs only once when the Arduino is powered ON or reset. It is used to initialise pin modes, variables, and libraries.

loop() function: It runs continuously until the Arduino is powered OFF. The main working logic is written here. It behaves like a while(1) loop in microcontroller programming.

Comments: A line beginning with // is a comment. The compiler ignores it. Comments help humans understand the program.

4.4 Program 1: Blink Built-in LED

```
void setup() {
pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
digitalWrite(LED_BUILTIN, HIGH);
delay(1000);
digitalWrite(LED_BUILTIN, LOW);
delay(1000);
}
```

Line-by-line explanation:

- pinMode(LED_BUILTIN, OUTPUT); sets the built-in LED pin, pin 13, as an output.
- digitalWrite(LED_BUILTIN, HIGH); turns the LED ON by giving HIGH voltage.
- delay(1000); waits for 1000 milliseconds, i.e., 1 second.
- digitalWrite(LED_BUILTIN, LOW); turns the LED OFF.
- The loop repeats, so the LED blinks once every second.

Expected output: The built-in LED turns ON for 1 second, turns OFF for 1 second, and repeats.

4.5 Program 2: LED ON/OFF on Pin 8

```
void setup() {
pinMode(8, OUTPUT);
}

void loop() {
digitalWrite(8, HIGH); // Turn on LED
delay(1000); // Wait 1 second
digitalWrite(8, LOW); // Turn off LED
delay(1000); // Wait 1 second
}
```

This program works like the blink program, but it uses digital pin 8 instead of the built-in LED pin. The LED must be connected to pin 8 through a suitable current-limiting resistor.

4.6 Arduino Data Types

Data type	Size on Uno	Range / use	Example
void	—	Function returns no value	void setup()
boolean / bool	1 byte	true or false	bool state = true;
byte	1 byte	0 to 255	byte m = 25;
word	2 bytes	0 to 65,535	word w = 1000;
int	2 bytes	-32,768 to 32,767	int counter = 32;
unsigned int	2 bytes	0 to 65,535	unsigned int counter = 60;
short	2 bytes	-32,768 to 32,767	short val = 13;
char	1 byte	One character; stored using ASCII	char cha = 'a';
unsigned char	1 byte	0 to 255	unsigned char y = 121;

Data type	Size on Uno	Range / use	Example
long	4 bytes	-2,147,483,648 to 2,147,483,647	long velocity = 102346;
unsigned long	4 bytes	0 to 4,294,967,295	unsigned long t = 101006;
float	4 bytes	Decimal numbers, about ±3.4028235E+38	float num = 1.352;
double	4 bytes on Uno	Same as float on ATmega-based boards	double num = 45.352;

Important character example: Characters are stored as ASCII numbers. The ASCII value of capital A is 65. Therefore, 'A' + 1 gives 66.

Common mistakes

- Do not confuse int with unsigned int: int can store negative numbers, unsigned int cannot.
- On Arduino Uno, double has the same size and precision as float.
- A byte cannot store negative numbers.
- Use unsigned long for time values returned by millis().

Exam points

- setup() runs once; loop() runs repeatedly.
- Verify compiles the code; Upload compiles and transfers it to the board.
- Comments start with //.
- int on Uno is 16-bit.
- float and double are both 4 bytes on Uno.
- Serial communication commonly uses 9600 baud.

5. Arduino Functions

5.1 Digital Functions

Digital functions work with two states: HIGH and LOW. HIGH usually means 5 V, and LOW usually means 0 V.

pinMode()

Purpose: Configures a pin as input or output. It is normally used inside setup().

```
pinMode(pin, mode);
```

- pin: Arduino pin number.
- mode: INPUT, OUTPUT, or INPUT_PULLUP.
- Example: pinMode(13, OUTPUT);

digitalWrite()

Purpose: Writes HIGH or LOW to a digital output pin.

```
digitalWrite(pin, value);
```

- pin: Arduino pin number.
- value: HIGH or LOW.
- HIGH means logic 1 and gives 5 V on the configured pin.
- Example: digitalWrite(13, HIGH);

digitalRead()

Purpose: Reads the value of a digital input pin. It returns HIGH or LOW.

```
digitalRead(pin);
```

```
void setup() {
pinMode(10, INPUT); // Pin 10 as input
}

void loop() {
int data = digitalRead(10); // Read pin 10
}
```

5.2 Program: Obstacle Sensor

Aim: Read an obstacle sensor connected to pin 2. If an obstacle is detected, print a message and turn ON the LED at pin 13.

```
int obstacle = 0;

void setup() {
pinMode(2, INPUT);
pinMode(13, OUTPUT);
Serial.begin(9600);
}

void loop() {
obstacle = digitalRead(2);
if (obstacle == LOW) {
Serial.print("Obstacle is present");
digitalWrite(13, HIGH);
} else {
```

```
digitalWrite(13, LOW);
}
delay(1000);
}
```

Working:

1. Pin 2 is set as input because the sensor gives a digital signal.
2. Pin 13 is set as output because it controls an LED.
3. Serial communication starts at 9600 baud.
4. The program reads pin 2 in every loop.
5. If the sensor output is LOW, it prints “Obstacle is present” and turns the LED ON.
6. Otherwise, it turns the LED OFF.
7. The program waits 1 second and repeats.

5.3 Analog Functions

analogRead()

Purpose: Reads the analog voltage from an analog input pin and converts it into a digital number.

```
analogRead(pin);
```

- Use pins A0 to A5 on Arduino Uno.
- The Uno has a multichannel 10-bit ADC.
- It maps 0 V to 0 and the operating voltage, normally 5 V, to 1023.
- Example: int value = analogRead(A0);

Program: Gas sensor reading

```
int gas_level;

void setup() {
pinMode(A0, INPUT);
Serial.begin(9600);
}

void loop() {
gas_level = analogRead(A0);
Serial.print("Gas Level : ");
Serial.println(gas_level);
delay(500);
}
```

Working: The gas sensor is connected to A0. Every 500 milliseconds, the Arduino reads its analog value, prints “Gas Level :” followed by the value, and moves to a new line on the Serial Monitor.

analogWrite()

Purpose: Writes a PWM signal to a PWM-capable pin. It is used to control LED brightness or motor speed.

```
analogWrite(pin, value);
```

- On Arduino Uno, PWM pins are 3, 5, 6, 9, 10, and 11.
- The value ranges from 0 to 255.

- 0 means always OFF; 255 means always ON.
- The pin produces a rectangular PWM wave with the selected duty cycle.

Program: Potentiometer controls LED brightness

```
int ledPin = 9;
int analogPin = 3;
int val = 0;

void setup() {
pinMode(ledPin, OUTPUT);
}

void loop() {
val = analogRead(analogPin);
analogWrite(ledPin, val / 4);
}
```

Working: analogRead() gives a value from 0 to 1023, but analogWrite() accepts only 0 to 255. Therefore, the reading is divided by 4 before writing it to the LED pin. This changes LED brightness according to the potentiometer position.

5.4 Time Functions

Function	Purpose	Unit	Important point
delay(ms)	Pauses the program	Milliseconds	1000 ms = 1 second
delayMicroseconds(us)	Pauses the program	Microseconds	Used for very short delays
millis()	Returns time since program started	Milliseconds	Overflows after about 50 days
micros()	Returns time since program started	Microseconds	Overflows after about 70 minutes

```
unsigned long time1;
unsigned long time2;

void setup() {
Serial.begin(9600);
}

void loop() {
time1 = millis();
time2 = micros();
Serial.println(time1); // milliseconds
Serial.println(time2); // microseconds
delay(1000);
}
```

5.5 Serial Communication Functions

Serial communication sends data one bit at a time between Arduino and a computer or another device. On Arduino Uno, pins 0 and 1 are used for RX and TX respectively.

Serial.begin()

```
Serial.begin(baud_rate);
```

It starts serial communication. The baud rate is the speed in bits per second. A common value is 9600.

```
Serial.begin(9600);
```

Serial.print() and Serial.println()

- Serial.print(val) prints data without moving to a new line.
- Serial.println(val) prints data and then moves to a new line.
- Both can print text, numbers, and formatted values.
- Formats include DEC, HEX, OCT, and BIN.

```
int analogValue = 0;

void setup() {
Serial.begin(9600);
}

void loop() {
analogValue = analogRead(A0);
Serial.println(analogValue); // Decimal
Serial.println(analogValue, DEC); // Decimal
Serial.println(analogValue, HEX); // Hexadecimal
Serial.println(analogValue, OCT); // Octal
Serial.println(analogValue, BIN); // Binary
delay(10);
}
```

Program: LDR sensor interfacing

```
void setup() {
pinMode(A0, INPUT);
Serial.begin(9600);
}

void loop() {
int light_intensity = analogRead(A0);
if (light_intensity > 800) {
Serial.println("Darker");
} else if (light_intensity < 500) {
Serial.println("Too Bright");
}
delay(1000);
}
```

Working: The LDR gives an analog value on A0. If the reading is greater than 800, the program prints “Darker”. If it is less than 500, it prints “Too Bright”. The program checks the light level once every second.

5.6 Math Functions

Function	Syntax	Purpose	Example
map()	map(value, fromLow, fromHigh, toLow, toHigh)	Converts a number from one range to another	map(val, 0, 1023, 0, 255)
max()	max(x, y)	Returns the larger value	max(sensVal, 20)
min()	min(x, y)	Returns the smaller value	min(sensVal, 100)
pow()	pow(base, exponent)	Calculates base raised to exponent	pow(x, y)
sq()	sq(x)	Returns the square of x	sq(4) gives 16
sqrt()	sqrt(x)	Returns the square root of x	sqrt(4) gives 2

map() example

```
void setup() {
pinMode(A0, INPUT);
}

void loop() {
int val = analogRead(A0);
val = map(val, 0, 1023, 0, 255);
analogWrite(9, val);
}
```

This converts the ADC range 0–1023 into the PWM range 0–255. It is a better method than dividing by 4 because map() clearly shows both ranges.

5.7 Advanced I/O Functions

pulseIn()

Purpose: Measures the duration of a HIGH or LOW pulse on a pin. It returns the pulse length in microseconds. If no complete pulse is received within the timeout, it returns 0.

```
pulseIn(pin, value);
pulseIn(pin, value, timeout);
```

```
int pin = 7;
unsigned long duration;

void setup() {
Serial.begin(9600);
pinMode(pin, INPUT);
}

void loop() {
duration = pulseIn(pin, HIGH);
Serial.println(duration);
}
```

random() and randomSeed()

- random(max) generates a pseudo-random number from 0 to max-1.
- random(min, max) generates a number from min to max-1.
- randomSeed() initialises the pseudo-random number generator.
- Using randomSeed(analogRead(A0)) gives a changing seed because an unconnected analog pin reads electrical noise.

```
long ranNumber;

void setup() {
Serial.begin(9600);
randomSeed(analogRead(A0));
}

void loop() {
ranNumber = random(300); // 0 to 299
Serial.println(ranNumber);
ranNumber = random(10, 20); // 10 to 19
Serial.println(ranNumber);
delay(50);
}
```

Common mistakes

- analogRead() returns 0–1023, but analogWrite() accepts 0–255.

- analogWrite() works only on PWM pins.
- digitalRead() returns HIGH or LOW, not a voltage value.
- random(10, 20) includes 10 but not 20.
- Use unsigned long for millis() and micros() values.

Exam points

- pinMode(), digitalWrite(), and digitalRead() are digital functions.
- analogRead(), analogWrite(), and analogReference() are analog functions.
- delay(), delayMicroseconds(), millis(), and micros() are time functions.
- Serial.begin(), Serial.print(), and Serial.println() are serial functions.
- map(), max(), min(), pow(), sq(), and sqrt() are math functions.
- pulseIn(), random(), and randomSeed() are advanced I/O functions.

6. ESP8266 and NodeMCU

Note: The original presentation lists “Introduction to ESP8266 – NodeMCU” in its outline, but the supplied extracted material does not include detailed NodeMCU teaching slides. The following is essential background added for completeness; verify details against your class notes if your exam includes NodeMCU.

6.1 What is ESP8266?

Simple definition: ESP8266 is a low-cost Wi-Fi microcontroller chip used in Internet of Things projects. It can connect to a Wi-Fi network and communicate over the Internet.

Unlike a basic Arduino Uno, the ESP8266 has built-in Wi-Fi. Therefore, it is useful for projects such as home automation, weather monitoring, smart plugs, and IoT data logging.

6.2 What is NodeMCU?

Simple definition: NodeMCU is an open-source development board based on the ESP8266 Wi-Fi chip.

Just as Arduino Uno makes the ATmega328P easy to use, NodeMCU makes the ESP8266 easy to use by adding USB, power regulation, and accessible pins.

- It includes Wi-Fi capability.
- It can be programmed using the Arduino IDE.
- It is commonly used for IoT applications.
- It provides GPIO pins for sensors and actuators.
- It supports serial communication for programming and debugging.

6.3 Arduino Uno vs NodeMCU

Feature	Arduino Uno	NodeMCU / ESP8266
Main chip	ATmega328P	ESP8266
Built-in Wi-Fi	No	Yes
Common use	Basic embedded control	IoT and Wi-Fi projects
Programming	Arduino IDE	Arduino IDE or other ESP8266 tools
Typical project	LED control, sensor reading	Sending sensor data to a cloud server

Exam points

- ESP8266 is a Wi-Fi microcontroller chip.
- NodeMCU is a development board based on ESP8266.
- NodeMCU is suitable for IoT applications.
- The presentation includes ESP8266–NodeMCU in the Unit 2 outline.

7. Final Revision Pack

7.1 Chapter-wise Revision

Microcontroller basics: A microcontroller is a single-chip computer with CPU, memory, clock, reset, and I/O. It is used in embedded systems.

Memory: Harvard uses separate program and data memory; Von Neumann uses common memory. ROM/Flash stores programs; RAM stores temporary data; EEPROM stores small permanent data.

Signals: Analog signals are continuous; digital signals are discrete. ADC converts analog to digital using sampling, quantization, and encoding.

Arduino Uno: It uses ATmega328P, has 14 digital pins, 6 analog inputs, 6 PWM pins, 10-bit ADC, 32 KiB Flash, 2 KiB SRAM, and 1 KiB EEPROM.

Programming: setup() runs once and loop() runs repeatedly. Use pinMode(), digitalWrite(), digitalRead(), analogRead(), analogWrite(), delay(), millis(), and Serial functions.

ESP8266/NodeMCU: ESP8266 is a Wi-Fi microcontroller; NodeMCU is its development board.

7.2 Important Definitions

Term	Definition
Microcontroller	A single-chip computer used to control embedded systems
Microprocessor	A processor that requires external memory and I/O components
Harvard architecture	Memory architecture with separate program and data memory
Von Neumann architecture	Memory architecture with common program and data memory
Firmware	Permanent software stored in read-only memory
Bootloader	Small program that helps load the main program into a microcontroller
ADC	Analog-to-Digital Converter; converts analog voltage into digital binary values
PWM	Pulse Width Modulation; produces an average analog-like output using digital pulses
Sketch	An Arduino program
Baud rate	Serial communication speed in bits per second
ICSP	In-Circuit Serial Programming; used to program microcontroller chips directly

7.3 Important Formulas and Syntax

```
ADC levels = 2^n, where n is the number of ADC bits.

Step size = Reference voltage / Number of levels

Input voltage = (ADC reading / 1023) × Reference voltage

pinMode(pin, INPUT or OUTPUT);

digitalWrite(pin, HIGH or LOW);
```

```
int x = digitalRead(pin);
int x = analogRead(A0); // 0 to 1023
analogWrite(pin, 0 to 255); // PWM pins only
delay(milliseconds);
Serial.begin(9600);
Serial.print(value); / Serial.println(value);
map(value, fromLow, fromHigh, toLow, toHigh);
```

7.4 Comparison Tables

Concept	Arduino Uno	NodeMCU
Microcontroller	ATmega328P	ESP8266
Wi-Fi	Not built in	Built in
Best use	Basic control and learning	IoT and Internet-connected projects
Programming environment	Arduino IDE	Arduino IDE / ESP8266 tools
Function	**Returns / action**	**Range**
digitalRead()	HIGH or LOW	Two states
analogRead()	Digital ADC value	0 to 1023
analogWrite()	Writes PWM value	0 to 255
millis()	Time since program start	Milliseconds
micros()	Time since program start	Microseconds

7.5 Last-Minute Checklist

- Can you draw and explain the microcontroller block diagram?
- Can you differentiate Harvard and Von Neumann architecture?
- Can you differentiate microcontroller and microprocessor?
- Can you explain analog signal, digital signal, and ADC?
- Do you know the ADC formula and the 10-bit ADC range?
- Can you label the main parts of Arduino Uno?
- Do you know Uno pin counts, PWM pins, and memory sizes?
- Can you explain setup() and loop()?
- Can you write LED blink, obstacle-sensor, gas-sensor, and potentiometer-LED programs?
- Do you know the purpose of every Arduino function covered in Unit 2?

7.6 Practice Questions

Short-answer questions

- Define a microcontroller.
- What is the difference between RAM and Flash memory?
- What is an ADC and why is it needed?

• State two differences between analog and digital signals.

• What is the function of setup() in Arduino?

• What is a bootloader?

• List the PWM pins of Arduino Uno.

• What is the range of analogRead() on Arduino Uno?

Short answers

• A microcontroller is a single-chip computer containing CPU, memory, clock, reset, and I/O circuits.

• RAM stores temporary data and loses it when power is removed; Flash stores program code permanently.

• An ADC converts analog voltage into digital binary values because microcontrollers understand digital data only.

• Analog signals are continuous; digital signals have fixed values such as 0 and 1.

• setup() runs once at power-ON or reset and is used for initialisation.

• A bootloader is a small program that allows a microcontroller to receive a new program through serial/USB.

• PWM pins are 3, 5, 6, 9, 10, and 11.

• analogRead() returns 0 to 1023.

Long-answer questions

• Explain the basic block diagram of a microcontroller with a neat labelled diagram.

• Compare microcontroller and microprocessor.

• Explain Harvard and Von Neumann memory architectures with diagrams.

• Explain analog-to-digital conversion and ADC resolution.

• Describe the architecture and specifications of Arduino Uno / ATmega328P.

• Explain the Arduino IDE and the process of compiling and uploading a sketch.

• Explain Arduino data types with examples.

• Write an Arduino program to blink an LED and explain each line.

• Write an Arduino program to read a gas sensor and display the value on the Serial Monitor.

• Explain digital, analog, time, serial, math, and advanced I/O functions in Arduino.

MCQs

Question	A	B	C	D	Answer	Explanation
1. Which microcontroller is used in Arduino Uno?	ATmega2560	ATmega328P	ESP8266	8051	B	Arduino Uno is based on the ATmega328P microcontroller.
2. How many digital I/O pins does Arduino Uno have?	6	10	14	20	C	The Uno has 14 digital I/O pins.
3. Which pins on Arduino Uno support PWM?	0, 1, 2, 3, 4, 5	3, 5, 6, 9, 10, 11	A0 to A5	8, 9, 10, 11, 12, 13	B	PWM pins are marked with (~): 3, 5, 6, 9, 10, and 11.

Question	A	B	C	D	Answer	Explanation
4. What is the ADC resolution of Arduino Uno?	8-bit	10-bit	12-bit	16-bit	B	The Uno has a 10-bit ADC.
5. What values does analogRead() return?	0 to 255	0 to 512	0 to 1023	0 to 4095	C	A 10-bit ADC returns 0 to 1023.
6. Which function runs only once in an Arduino sketch?	loop()	setup()	main()	start()	B	setup() runs once at startup or reset.
7. Which architecture uses separate program and data memory?	Von Neumann	Princeton	Harvard	Harvard and Von Neumann	C	Harvard architecture uses separate memories.
8. Which memory stores the Arduino program permanently?	RAM	SRAM	Flash	Cache	C	Flash memory stores the program code.
9. Which function is used to read a digital pin?	analogRead()	digitalRead()	digitalWrite()	pinMode()	B	digitalRead() reads HIGH or LOW from a digital pin.
10. What is the range of analogWrite() on Arduino Uno?	0 to 100	0 to 255	0 to 1023	0 to 65535	B	PWM values range from 0 to 255.
11. Which function measures pulse duration in microseconds?	delay()	millis()	pulseIn()	map()	C	pulseIn() measures pulse duration in microseconds.
12. Which chip acts as the USB-TTL interface on Arduino Uno?	ATmega328P	ATmega16U2	ATmega2560	ESP8266	B	The ATmega16U2 performs USB-to-TTL conversion.
13. Which data type stores true or false?	int	char	boolean	float	C	boolean stores true or false.
14. Which function converts 0–1023 to 0–255?	max()	min()	map()	sqrt()	C	map() re-maps one numeric range to another.
15. What does IDE stand for?	Internet Development Engine	Integrated Development Environment	Input Device Editor	Internal Data Encoder	B	IDE means Integrated Development Environment.

Answer key

1-B, 2-C, 3-B, 4-B, 5-C, 6-B, 7-C, 8-C, 9-B, 10-B, 11-C, 12-B, 13-C, 14-C, 15-B